

# Kesher Learned Taste Implementation Dossier

## Executive answer

**VERIFIED:** The selected Kesher repo already contains the product surfaces and policy posture that point in the right direction: a finite, intentional **Daily Picks** flow; a separate **Explore** surface with hard filters and soft preferences; an **AI & Trust Hub**; a **Private Taste Profile** that can be edited or reset; and explicit red lines against photo-based inference, attractiveness scoring, auto-chatting, and hidden manipulation. It also now routes AI calls through server endpoints rather than exposing a client-side API key, so some older planning docs in the repo are already partially outdated by the current code. [1](#) [2](#) [3](#) [4](#) [5](#) [6](#) [7](#) [8](#) [9](#)

**VERIFIED:** The current Learned Taste implementation is not production-ready as a privacy-preserving taste system. In the current state, the runtime interaction model collects only a small set of interaction strings, the taste profile object is still a thin bundle of lists plus two numeric weights, and the schema lacks provenance, confidence, recency, decay, expiry, and granular user controls. The feature registry also advertises `dwelt_time` as an input even though the live application state shown in the repo does not actually track it. [10](#) [11](#) [12](#) [13](#) [14](#) [15](#) [16](#) [17](#)

**INFERRED:** Kesher should therefore ship Learned Taste as a **strictly private, user-auditable, low-intensity personalization layer** that begins with explicit controls and only later incorporates a **small, clearly disclosed, first-party behavioral layer**. The system should rank on explicit preferences first, weak behavioral hints second, and only convert repeated hints into stable learned patterns after thresholding, decay, contradiction handling, and user visibility. This follows both recommender-systems evidence that implicit feedback is informative but noisy and biased, and privacy-law principles that require minimization, transparency, storage limits, and by-default restraint. [18](#)

**HEURISTIC:** The best concrete launch shape is:

1. **Launch v1 with explicit signals only** for learning: hard filters, soft preferences, recommendation mode, manual profile edits, and the very intentional “More like this” / “Less like this” controls.
2. **Do not use** message text, photo embeddings, attractiveness proxies, inferred religiosity, inferred ethnicity, inferred politics, inferred sexual orientation, cross-app data, precise location trails, contact graphs, payment status, or hidden desirability/popularity metrics.
3. **Treat likes and passes as weak hints only**, not direct preference statements.
4. **Keep passive behavioral learning off by default**; let users opt in later with a clear “Improve my Daily Picks from my activity” control.
5. **Show every learned pattern back to the user** with provenance, confidence, and expiry, and let them edit, pause, export, or delete it.
6. **Do not ship the current repo’s Learned Taste logic as-is** until auth, retention, provenance, and editability are complete. [19](#) [3](#) [5](#)

**Assumptions:** I do not see additional uploaded Kesher packs in this session beyond the selected repo, so this dossier treats the repo as the operative first-party source of product truth.

**Uncertainties:** Jurisdiction-specific legality for using explicitly provided observance or other religion-linked data in matching still requires counsel review.

**Conclusion:** Keshar should build **subtle, inspectable, revocable relevance**, not hidden psychological profiling.

## Definitions and signal map

### Definitions and term resolution

**VERIFIED:** Modern recommender literature distinguishes **explicit feedback** from **implicit feedback**.

Explicit feedback is a direct statement of preference; implicit feedback is observed behavior that may correlate with preference but does not directly express it. In implicit-feedback systems, the signal is typically treated as a confidence-weighted indication, not as a clean ground-truth preference. Click and behavior signals are also subject to exposure, position, and context bias. Preferences themselves drift over time, so temporal handling matters. <sup>20</sup>

For Keshar, the right operational definitions are:

#### **EXPLICIT PREFERENCE — INFERRED**

A user intentionally states or edits a preference in a way that plainly communicates meaning. This includes hard filters, soft preferences, recommendation mode selection, manual edits inside the Taste Profile, and explicit “More like this” / “Less like this” actions. These should be high-confidence, user-visible, and directly editable. This aligns with the current product surfaces already present in the repo. <sup>2 3 14</sup>

#### **WEAK BEHAVIORAL HINT — INFERRED**

A user action that may imply preference but is ambiguous. This includes like, pass, revisit, profile-detail open, save/bookmark if implemented later, or a very coarse capped dwell bucket if Keshar ever adds it. These should be low-confidence, high-decay, and never shown to the user as settled facts unless repeated. Evidence says this caution is necessary because behavioral signals are informative but biased and context-dependent. <sup>21</sup>

#### **STABLE LEARNED PATTERN — HEURISTIC**

A preference candidate elevated from weak hints only after repeated, consistent evidence across time and contexts. This is not the same thing as a hard fact. It should be a small-weight ranking feature, visible to the user, easy to remove, and subject to decay and contradiction. Temporal-dynamics research strongly supports recency-aware handling rather than permanent accumulation. <sup>22</sup>

### Acceptable signals for ranking

**VERIFIED / INFERRED:** The following inputs are appropriate for Learned Taste or ranking, with different strengths.

#### **Direct, high-quality inputs**

- Hard filters the user intentionally sets, such as age range, max distance, and verified-only. These are explicit, understandable, and already modeled in the repo's discovery and filter surfaces. <sup>2 23</sup>

- Soft preferences the user intentionally sets or edits. These should remain editable and visible. <sup>2</sup>  
<sup>3</sup>
- Recommendation mode choice such as values-first, balanced, or chemistry-first. This belongs in the profile as an explicit weighting preference, not a hidden model choice. <sup>2</sup>
- “More like this” and “Less like this.” These are the best behavioral inputs because the meaning is unambiguous and user-intentional. The repo already uses them to call the taste-profile analyzer. <sup>14</sup>  
<sup>24</sup>
- Manual edits, locks, or removals from the Taste Profile. A user correction should override any behavioral inference. This follows rectification principles and the repo's existing editable profile direction. <sup>25</sup> <sup>3</sup>

### Cautious, lower-quality inputs

- Likes and passes inside Kesher's own discovery surfaces. These can inform low-confidence hints, but they should not be treated as explicit preference declarations because they are heavily influenced by presentation context and item availability. The current repo already stores likes and passes as interaction traces. <sup>26</sup> <sup>12</sup> <sup>13</sup>
- Repeated opens of full profile detail or repeated returns to similar profiles, if Kesher later adds them. These are acceptable only as weak hints and only when aggregated over multiple sessions.
- A **coarse dwell bucket** such as “looked briefly / looked carefully,” if Kesher ever adds it. This should be opt-in, capped, and used only as a tie-breaker or confidence modifier, not as a standalone preference claim. The current feature registry mentions dwell time, but the live interaction state in the repo does not actually capture it, which is good reason not to rush it. <sup>16</sup> <sup>11</sup>

### Signals that should never be used for ranking

**VERIFIED / INFERRED:** Kesher should explicitly ban the following from Learned Taste.

- **Photo-derived inference of attractiveness, religiosity, ethnicity, class, or personality.** The repo already forbids photo inference and attractiveness scoring; that red line should remain absolute. <sup>5</sup>  
<sup>4</sup>
- **Message text, message sentiment, response latency, or conversation style** for taste learning. These belong to safety or communication-assist surfaces, not hidden preference extraction. Using them for matching would feel invasive and causally ambiguous. The feature registry already excludes messages from Taste Profile inputs. <sup>16</sup>
- **Protected-trait inference** from behavior, especially religion, politics, health, race/ethnicity, sex life, or sexual orientation. Under GDPR Article 9, data revealing these categories is specially restricted; even where the user explicitly provides relevant information, inferring more from behavior is materially higher risk and should be avoided. <sup>27</sup>
- **Off-platform browsing, ad-tech data, third-party data brokers, contact graphs, address books, phone metadata, precise location history, or device fingerprinting.** These are unnecessary for Kesher's stated purpose and cut directly against data minimization and privacy-by-default. <sup>28</sup>
- **Safety reports, blocks, fraud flags, or moderation signals** as taste inputs. These may justify exclusion or safety intervention, but not preference learning. Safety and recommendation logic should stay separated.
- **Popularity or desirability scoring** such as “people like users who look like X,” premium status, reply rate, or hidden market-value metrics. That would violate both the repo's product truth and the anti-manipulation stance. <sup>5</sup> <sup>4</sup>

## Taste profile model design

**VERIFIED:** The current repo's Taste Profile schema consists of `hard_dealbreakers`, `soft_preferences`, `things_to_avoid`, `weights`, and `explanation`. It does not include provenance, confidence, recency, decay, expiry, contradiction handling, or user-lock semantics. That is the critical design gap. <sup>17</sup> <sup>29</sup>

**INFERRED:** Keshar should represent taste in **four layers**, not one flat object.

### Explicit layer

What the user knowingly set. This includes hard filters, soft preferences, verified-only rules, recommendation mode, and any manual edits. These are top priority, fully visible, and do not auto-expire unless the user chooses. If the user locks a value, the system should never overwrite it.

### Hint layer

Low-confidence behavioral aggregates, such as “recently tends to like profiles with long-form bios” or “recently tends to avoid long-distance profiles.” These should be:

- low weight,
- non-final,
- highly decayed,
- internally stored with counts and recency,
- not displayed as hard claims unless they stabilize.

### Learned layer

Stable patterns that graduate from the hint layer. Each item should have:

- a category,
- a value,
- a sensitivity tier,
- a provenance summary,
- a confidence score,
- support count,
- contradiction count,
- last updated time,
- decay half-life,
- expiry timestamp,
- ranking weight cap,
- edit/remove/lock state.

### Exclusion layer

Items or categories the user has said should not affect the profile. This mirrors patterns already used by Spotify <sup>30</sup>, Apple <sup>31</sup>, and Netflix <sup>32</sup>, each of which gives users a way to stop specific listening or viewing history from shaping recommendations. <sup>33</sup>

**HEURISTIC:** Keshar's internal fields should therefore include, at minimum:

- `category`
- `value`
- `source_class` such as explicit, hint, or learned
- `provenance_summary` such as "from 6 likes and 2 more-like-this actions in the last 30 days"
- `confidence` on a bounded scale
- `support_count` and `distinct_session_count`
- `last_confirmed_at`
- `decay_half_life_days`
- `expires_at`
- `user_locked`
- `user_hidden`
- `rank_weight_cap`
- `eligible_for_ranking` true or false
- `sensitivity_tier` such as normal, high-risk, never-learn

**HEURISTIC:** Category policy should be:

- **Hard filters:** explicit only, never learned.
- **Soft preferences:** explicit or learned.
- **Trust/safety preferences:** explicit or learned from many repeated choices, but always user-visible.
- **Interests and lifestyle tags:** learnable.
- **Distance flexibility:** explicit or learned only very cautiously.
- **Religion-linked or protected-trait-adjacent attributes:** explicit only, never behaviorally inferred.
- **Attractiveness or appearance abstractions:** never represented.

**HEURISTIC:** Decay and retention should be concrete.

- Raw behavioral events used for taste learning: retain for **30 days** in active form, then aggregate or delete.
- Hint-layer aggregates: half-life **21–30 days**, expire at **90 days** if not refreshed.
- Stable learned patterns: half-life **60–90 days**, expire at **180 days** unless refreshed or user-locked.
- Explicit preferences: persist until edited or deleted by the user.

This recommendation is supported by storage-limitation and by-default minimization principles, plus the evidence that preference drift is real and permanent accumulation is usually wrong. <sup>34</sup>

## User control surface and language

**VERIFIED:** The repo already points toward the right UI posture: the **Private Taste Profile** can be edited and reset; the Trust Hub says the profile is private and "never shared with other users"; and the Trust Hub already exposes red lines and feature controls. <sup>3</sup> <sup>4</sup>

**INFERRED:** Keshar should turn that into a full control surface with five user rights:

## View

A dedicated Taste Profile page should show three groups:

- **You set**
- **Learned from activity**
- **Ignored for learning**

Each learned item should show:

- what it is,
- why it was added,
- how confident the system is,
- when it was last updated,
- when it will expire if untouched.

## Edit

Each item should have **Edit**, **Lock**, and **Remove**. Manual edits should win over model updates.

## Pause

Kesher should add a **Stop behavioral updates** control. My recommendation is:

- **Passive behavioral learning: default off** until the user opts in.
- **Explicit feedback controls** like “More like this” / “Less like this”: remain usable regardless, because they are direct commands rather than passive surveillance.

This is the cleanest reconciliation between relevance and privacy-by-default. It also tracks the direction of mainstream control patterns: Apple Music lets users turn off listening history entirely, Spotify lets users exclude specific playlists or tracks from their taste profile, and Netflix lets users hide titles from viewing history so they stop affecting recommendations. <sup>35</sup>

## Export

Users should be able to export:

- their explicit preferences,
- their learned patterns,
- their exclusion list,
- event counts and timestamps,
- provenance summaries,
- decay and expiry metadata.

This is strongly supported by access and portability rights in GDPR Articles 15 and 20. <sup>36</sup>

## Delete

Offer two paths:

- **Reset learned taste** only, and
- **Delete all taste-learning data** including aggregates and retained raw events.

This follows rectification and erasure principles and should be simple, not maze-like. 37

**INFERRED:** The user-facing copy should stay subtle, concrete, and non-anthropomorphic. Recommended language:

- “Improve my Daily Picks from my activity.”
- “Use only my Keshet activity.”
- “You can pause this anytime.”
- “Not shared with other users.”
- “Learned from recent likes, passes, and ‘more like this’ actions.”
- “You can remove anything here or start fresh.”

Avoid language like:

- “We know your type.”
- “Our AI figured you out.”
- “Your hidden preferences.”
- “Your compatibility DNA.”
- “Based on your behavior across the web.”

The reason is not only tone; HCI and legal evidence both show that personalization becomes creepy when causal chains are opaque, over-personalized, or manipulative. 38

## Core findings and privacy risks

**VERIFIED:** Keshet’s own first-party product truth is strong. The repo’s AI policies state `NO_PHOTO_INFERENCE`, `NO_ATTRACTIVENESS_SCORING`, `PRIVATE_TASTE_ONLY`, and `NO_AUTO_SEND`; the Taste Profile system instruction says the assistant must never infer sensitive traits like race, ethnicity, or attractiveness; and the Trust Hub repeats the same red lines in user-facing language. Those red lines should be preserved without exception. 5 4

**VERIFIED:** The first major fault line is **representation quality**. The current schema is too thin for accountable personalization. A taste object without provenance, confidence, expiry, contradiction handling, or a user lock state is not auditable enough for a sensitive product context. 17

**VERIFIED / INFERRED:** The second fault line is **signal ambiguity**. The feature registry says Taste Profile may use likes, passes, and dwell time and excludes messages and real names, but the current application state actually records only likes, passes, more-like-this, and less-like-this strings, and only the more/less-like-this actions immediately trigger model refresh. That inconsistency is a privacy and product-risk signal in itself: the product has not yet clearly decided what behavioral evidence it wants to collect. 16 11 14

**VERIFIED / INFERRED:** The third fault line is **special-category creep**. In a dating context, seemingly harmless behavior can quickly imply religion, sexual preferences, family expectations, or health-related inferences. GDPR Article 9 treats personal data revealing religion, health, sex life, and sexual orientation as specially restricted. Even where Keshet uses explicit observance or intent fields, it should not behaviorally infer more than the user deliberately chose to disclose. 27

**VERIFIED / INFERRED:** The fourth fault line is **deceptive design**. Research and enforcement sources show that dark-pattern interfaces can manipulate preference expression, obscure user choice, and even produce backlash when users feel tricked. Recommender research specifically finds that creepiness rises when users cannot tell how the system knows something, and that this harms trust and platform attitudes. The safe lesson for Keshar is simple: no hidden optimization loops, no manipulative consent language, no burying of the pause/reset controls, and no “just trust us” explanations. <sup>39</sup>

**VERIFIED / INFERRED:** The fifth fault line is **backend readiness**. The current code does show a server-side AI proxy, which is a major improvement over the older planning documents, but the visible `server.ts` and `aiRoutes.ts` do not show auth middleware or rate limiting around taste-profile routes. That is acceptable for a demo, but not for durable preference data in production. <sup>8</sup> <sup>7</sup> <sup>40</sup>

## Verification experiments

**HEURISTIC:** Keshar should not validate Learned Taste only on relevance lift. It should validate **relevance, trust, and privacy cost together**.

### Experiment one: explicit-only versus explicit-plus-behavioral

Compare:

- explicit settings only,
- explicit settings plus “more/less like this,”
- explicit settings plus a narrow opt-in behavioral layer.

Success must require both:

- improved pick acceptance or save rate, and
- no meaningful increase in “this felt invasive” or “how did you know that?” responses.

This structure is evidence-based because implicit-feedback systems improve ranking but also raise creepiness and causal-ambiguity risk. <sup>41</sup>

### Experiment two: transparency comprehension

After showing a learned pattern, ask a one-tap question:

- “Does this explanation make sense?”

Target comprehension, not mere clickthrough. GDPR Articles 12–15 and Article 22’s logic-and-consequences disclosures make this especially important when profiling is involved. <sup>42</sup>

### Experiment three: protected-trait leakage audit

Run internal tests to determine whether the learned features can predict protected or high-risk categories better than chance. If they can, the representation is too revealing and must be pruned. This follows the AI-risk posture described by NIST <sup>43</sup> in the AI RMF, especially continuous govern-map-measure-manage discipline around harms to people and groups. <sup>44</sup>



### Experiment four: decay audit

Measure whether recommendations improve when old hints expire more aggressively. If old signals reduce satisfaction, shorten half-lives. Preference drift is not theoretical; it is a standard recommender-systems reality. <sup>45</sup>

### Experiment five: controls usability

Time how long it takes a user to:

- pause behavioral updates,
- remove a learned pattern,
- reset learned taste,
- export their profile,
- delete taste data.

If any of these paths feel materially harder than enabling personalization, Kesher is drifting into deceptive design territory. Guidance from the Federal Trade Commission <sup>46</sup> and the European Data Protection Board <sup>47</sup> strongly supports avoiding interfaces that impair autonomy, obscure choices, or make privacy-protective action harder than data-sharing action. <sup>48</sup>

## Launch recommendation

**VERIFIED / INFERRED:** Kesher should **not** launch the current Learned Taste implementation as a fully behavioral system.

### Recommended launch shape

#### Phase one

Ship:

- hard filters,
- soft preferences,
- recommendation mode,
- editable Taste Profile,
- “More like this” / “Less like this,”
- reset/export/delete,
- “Stop behavioral updates” control,
- server-authenticated storage only.

Do **not** ship:

- passive dwell learning,
- message-based learning,
- photo-based inference,
- protected-trait inference,
- off-platform data enrichment.

This phase gives Kesher most of the relevance gain with the least creepiness risk. It also stays closest to the repo's current product truth. 2 3 5

### Phase two

If Phase one performs well, add **optional, default-off passive hints** from first-party in-product actions only:

- likes,
- passes,
- revisits,
- maybe a coarse dwell bucket.

Require:

- explicit opt-in,
- visible learned-pattern cards,
- support thresholds,
- decay,
- easy pause and removal,
- no use of exact profile names in the standard UI.

### Phase three

Only after repeated trust-safe wins should Kesher consider a broader behavioral model, and even then it should remain inside its strict red lines.

### Concrete go / no-go decision

- **Go** for explicit and inspectable learning.
- **No-go** for hidden or passive profiling at launch.
- **No-go** for any system that infers religion, attractiveness, or other protected or intimate traits from behavior or photos.
- **No-go** for releasing without auth, retention controls, export/delete, and pause/reset.

That is the concrete recommendation.

## Open legal and policy questions

**UNKNOWN:** What is the lawful basis, jurisdiction by jurisdiction, for using explicitly provided observance or other religion-linked matching data, and what additional safeguards are required where GDPR or similar regimes apply? Article 9 makes this a high-priority review item. 49

**UNKNOWN:** Does Kesher's ranking function ever become sufficiently consequential that counsel would treat it as profiling with similarly significant effects under Article 22 in some jurisdictions or circumstances? If yes, more explicit explanation, contestation, and human-review design may be required. 50

**UNKNOWN:** What exact retention schedule should apply to raw interaction events versus aggregates, and what must be included in the export artifact versus withheld to protect the privacy of other users? Articles 13, 15, 17, and 20 matter here. <sup>51</sup>

**UNKNOWN:** Should explicit observance, intent, or value fields always remain explicit-only, or can any part of them be softly learned in some jurisdictions with explicit consent? My recommendation is still **no behavioral inference**, but this remains a policy decision requiring legal sign-off. <sup>49</sup>

**UNKNOWN:** When Kesher eventually supports broader gender or orientation matching settings, how will it separate explicit match-eligibility settings from prohibited inference about sex life or sexual orientation? This should be resolved before any expansion of Learned Taste. <sup>52</sup>

---

<sup>24</sup> DailyPicksScreen.tsx

<https://github.com/akivagoldstein61-ui/Google-ai-studio/blob/main/src/features/discovery/DailyPicksScreen.tsx>

<sup>2</sup> <sup>47</sup> ExploreScreen.tsx

<https://github.com/akivagoldstein61-ui/Google-ai-studio/blob/main/src/features/discovery/ExploreScreen.tsx>

<sup>3</sup> <sup>46</sup> PrivateTasteProfile.tsx

<https://github.com/akivagoldstein61-ui/Google-ai-studio/blob/main/src/features/settings/PrivateTasteProfile.tsx>

<sup>4</sup> AITrustHub.tsx

<https://github.com/akivagoldstein61-ui/Google-ai-studio/blob/main/src/features/settings/AITrustHub.tsx>

<sup>5</sup> policies.ts

<https://github.com/akivagoldstein61-ui/Google-ai-studio/blob/main/src/ai/policies.ts>

<sup>6</sup> aiService.ts

<https://github.com/akivagoldstein61-ui/Google-ai-studio/blob/main/src/services/aiService.ts>

<sup>7</sup> aiRoutes.ts

<https://github.com/akivagoldstein61-ui/Google-ai-studio/blob/main/server/aiRoutes.ts>

<sup>8</sup> server.ts

<https://github.com/akivagoldstein61-ui/Google-ai-studio/blob/main/server.ts>

<sup>9</sup> vite.config.ts

<https://github.com/akivagoldstein61-ui/Google-ai-studio/blob/main/vite.config.ts>

<sup>29</sup> <sup>43</sup> AppContext.tsx

<https://github.com/akivagoldstein61-ui/Google-ai-studio/blob/main/src/context/AppContext.tsx>

<sup>16</sup> featureRegistry.ts

<https://github.com/akivagoldstein61-ui/Google-ai-studio/blob/main/src/ai/featureRegistry.ts>

<sup>17</sup> schemas.ts

<https://github.com/akivagoldstein61-ui/Google-ai-studio/blob/main/src/ai/schemas.ts>

<sup>18</sup> <sup>19</sup> <sup>20</sup> <sup>21</sup> <sup>30</sup> <sup>41</sup> [https://www.researchgate.net/publication/](https://www.researchgate.net/publication/220765111_Collaborative_Filtering_for_Implicit_Feedback_Datasets)

220765111\_Collaborative\_Filtering\_for\_Implicit\_Feedback\_Datasets

[https://www.researchgate.net/publication/220765111\\_Collaborative\\_Filtering\\_for\\_Implicit\\_Feedback\\_Datasets](https://www.researchgate.net/publication/220765111_Collaborative_Filtering_for_Implicit_Feedback_Datasets)

22 31 45 <https://cacm.acm.org/research/collaborative-filtering-with-temporal-dynamics/>  
<https://cacm.acm.org/research/collaborative-filtering-with-temporal-dynamics/>

23 [index.ts](#)  
<https://github.com/akivagoldstein61-ui/Google-ai-studio-/blob/main/src/types/index.ts>

25 27 28 34 36 37 42 49 50 51 52 <https://eur-lex.europa.eu/legal-content/EN/TXT/?qid=1644497630905&uri=CELEX%3A02016R0679-20160504>  
<https://eur-lex.europa.eu/legal-content/EN/TXT/?qid=1644497630905&uri=CELEX%3A02016R0679-20160504>

26 <https://pure.psu.edu/en/publications/accurately-interpreting-clickthrough-data-as-implicit-feedback>  
<https://pure.psu.edu/en/publications/accurately-interpreting-clickthrough-data-as-implicit-feedback>

32 48 <https://www.ftc.gov/news-events/events/2021/04/bringing-dark-patterns-light-ftc-workshop>  
<https://www.ftc.gov/news-events/events/2021/04/bringing-dark-patterns-light-ftc-workshop>

33 <https://support.spotify.com/il-en/article/your-taste-profile/>  
<https://support.spotify.com/il-en/article/your-taste-profile/>

35 <https://support.apple.com/en-ae/guide/iphone/iph2b1748696/ios>  
<https://support.apple.com/en-ae/guide/iphone/iph2b1748696/ios>

38 <https://interactivesystems.info/system/pdfs/934/original/RecSys-19.pdf?1574934623=>  
<https://interactivesystems.info/system/pdfs/934/original/RecSys-19.pdf?1574934623=>

39 <https://academic.oup.com/jla/article-abstract/13/1/43/6180579>  
<https://academic.oup.com/jla/article-abstract/13/1/43/6180579>

40 [07\\_updated\\_plan\\_vnext.md](#)  
[https://github.com/akivagoldstein61-ui/Google-ai-studio-/blob/main/docs/claude-import-refresh/07\\_updated\\_plan\\_vnext.md](https://github.com/akivagoldstein61-ui/Google-ai-studio-/blob/main/docs/claude-import-refresh/07_updated_plan_vnext.md)

44 <https://www.nist.gov/itl/ai-risk-management-framework>  
<https://www.nist.gov/itl/ai-risk-management-framework>